

Flink 原理详解：第 4 讲

Window 窗口机制：把无限流切成可以计算的有限问题

Flink 原理详解 · 第 4 讲

本讲核心问题：Watermark 只是在告诉 Flink “事件时间大致走到哪里”。真正决定“哪些事件属于同一批、什么时候触发、触发后怎么算”的，是 Window Assigner、Trigger、State 与 Window Function 的协作。

学习目标

- 理解为什么无界流必须通过 Window 才能做“每 5 分钟”“最近 1 小时”这类聚合。
- 能区分 Tumbling、Sliding、Session 三类窗口，并知道各自适合的现实场景。
- 彻底区分 Window Assigner、Trigger、Watermark、Window State、Window Function 的职责。
- 理解 Event Time 窗口的生命周期、Allowed Lateness 与迟到数据。
- 理解增量聚合与全量窗口函数的性能差异，并能设计一个简单实时业务窗口。

无界流：事件不断到来，没有天然“结尾”

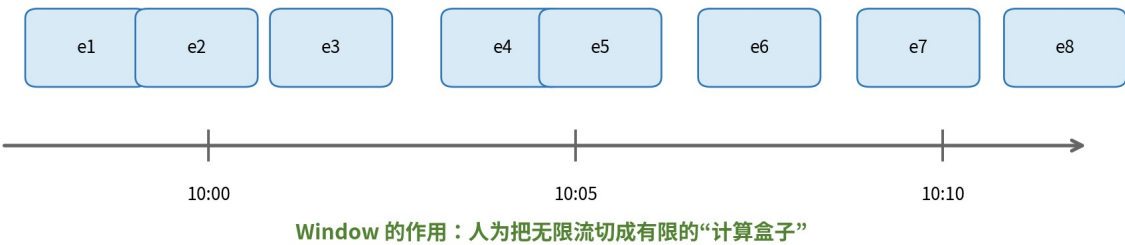


图 1: Window 的根本目的不是“画时间段”，而是给无界流制造有限计算边界。

1. 为什么流处理必须有 Window？

在批处理中，一个文件天然有开始和结束；读取完最后一条记录，就可以统计总和、平均值、最大值。但实时流可能永远不会结束。

问题：如果你对一条永远不结束的订单流直接问“总销售额是多少”，答案会永远变化，系统也永远等不到“最终完成”。

所以流处理中经常把问题改写成有限问题，例如：“每 5 分钟销售额”“最近 1 小时活跃用户数”“每个会话的点击次数”。Window 就是把无限流切成一个个有限的计算范围。

业务问题	窗口化后的计算单位	典型窗口
每 5 分钟销售额	10:00-10:05、10:05-10:10……	Tumbling

每分钟更新最近 10 分钟请求量	09:51-10:01、09:52-10:02……	Sliding
一次用户访问会话有多少操作	连续活动属于同一段，长时间无操作则结束	Session

记忆：Window 本质上是在回答：“我准备把哪一批事件放在一起算？”

2. 三种最常见窗口

Tumbling Window（滚动窗口）



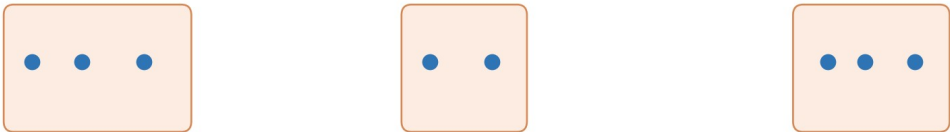
固定长度、不重叠；一条事件只属于一个窗口

Sliding Window（滑动窗口）



窗口长度 > 滑动步长时会重叠；一条事件可能进入多个窗口

Session Window（会话窗口）



边界由“不活跃间隔”决定；没有固定开始/结束，可能发生窗口合并

图 2: Tumbling、Sliding、Session Window 的直观区别。

2.1 Tumbling Window：滚动窗口

滚动窗口长度固定、彼此不重叠。一条事件只属于一个窗口。比如“每 5 分钟统计一次城市销售额”。

5 分钟滚动窗口

10:00 - 10:05
10:05 - 10:10
10:10 - 10:15

它最适合固定周期报表、实时大屏、每分钟/每小时统计等场景。

2.2 Sliding Window：滑动窗口

滑动窗口由两个参数决定：窗口长度 size 和滑动步长 slide。当 $size > slide$ 时，窗口会重叠，一条事件可能同时属于多个窗口。

例：每 5 分钟更新一次“最近 10 分钟”统计

窗口长度 = 10 分钟

滑动步长 = 5 分钟

09:50 - 10:00

09:55 - 10:05

10:00 - 10:10

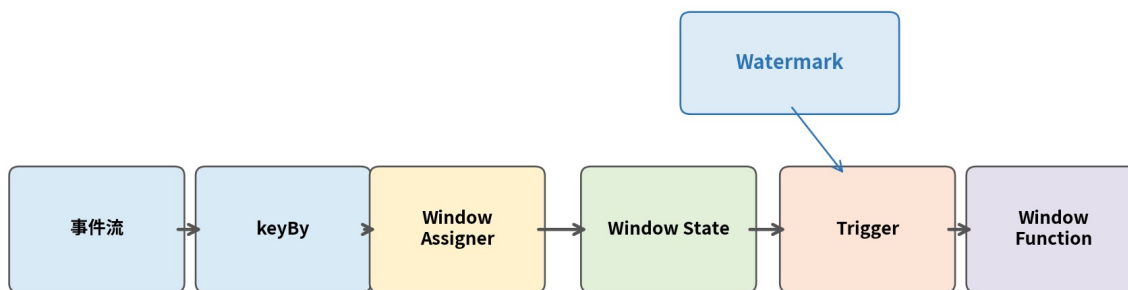
性能提醒：窗口重叠意味着同一事件可能参与多个窗口。窗口越大、步长越小，状态和计算成本通常越高。

2.3 Session Window：会话窗口

Session Window 没有预先固定的开始和结束边界，而是用“多久没有新事件”判断会话是否结束。比如用户连续点击、浏览、购买都属于同一次会话；超过 30 分钟没有活动，再来的事件进入新会话。

Session Window 的特殊之处是：新事件到来后，原本分开的窗口可能需要合并，因此它比固定时间窗口的状态管理更复杂。

3. Window 不是一个组件，而是一套协作机制



Assigner 决定“属于哪个窗口” | Trigger 决定“什么时候计算” | Function 决定“怎么算”

图 3: Window 计算链条。Watermark 是时间进度信号，不负责定义窗口边界。

组件	核心问题	通俗解释
Window Assigner	事件属于哪个窗口？	分班老师：把事件放入某个时间盒子
Window State	窗口里记住什么？	临时账本：保存元素或聚合器状态
Trigger	什么时候计算/输出？	闹钟：决定何时叫 Window Function 工作
Window Function	窗口怎么算？	计算员：sum、avg、TopN、业务逻辑
Watermark	Event Time 推进到哪里？	时间进度线：给 EventTimeTrigger 提供依据

最重要的区分：Window Assigner 决定“属于哪里”；Trigger 决定“什么时候算”；Window Function 决定“怎么算”；Watermark 只提供 Event Time 的进度证据。

4. Window Assigner：到底是谁决定 5 分一口？

答案是 Window Assigner。比如 5 分钟 Tumbling Event-Time Window，会根据每条事件自身的 Event Time，把它映射到某个时间区间。

概念示例

```
# 概念代码：不同 PyFlink 版本在 Time / Duration 类型上可能略有差异
stream \
    .key_by(lambda x: x.city) \
    .window(TumblingEventTimeWindows.of(5_minutes)) \
    .aggregate(...)
```

假设窗口边界按整 5 分钟对齐：

事件 Event Time	所属窗口
10:00:12	[10:00, 10:05)

10:04:59	[10:00, 10:05)
10:05:00	[10:05, 10:10)
10:09:31	[10:05, 10:10)

边界细节：时间窗口通常可理解为左闭右开 [start, end)。因此 10:05:00 不再属于 10:00-10:05，而进入下一个窗口。

5. Trigger：窗口什么时候真正计算？

Assigner 只负责分配，并不等于“立刻输出结果”。Trigger 负责判断何时让窗口函数执行。对常见 Event-Time Window，默认 EventTimeTrigger 会随着 Watermark 推进，在 Watermark 越过窗口结束时间时触发。

Event Time 窗口的直觉

```
Window = [10:00, 10:05)
事件陆续到达 -> 更新窗口状态
Watermark = 10:03 -> 不触发
Watermark 越过 10:05 -> FIRE -> 计算并输出
```

Trigger 的结果可以理解为四种动作：CONTINUE（继续等）、FIRE（计算并输出）、PURGE（清理窗口内容）、FIRE_AND_PURGE（计算输出并清理）。初学阶段不必自己写 Trigger，但要知道“触发”和“清理”是两件不同的事。

常见误区：“Watermark 到了就把窗口删掉”并不准确。默认 Trigger 可以 FIRE 而不立即 PURGE；窗口真正何时被删除，还与 Allowed Lateness 等生命周期规则有关。

6. Window 生命周期与 Allowed Lateness

Window 生命周期（Event Time）



图 4: Event-Time Window 的典型生命周期。

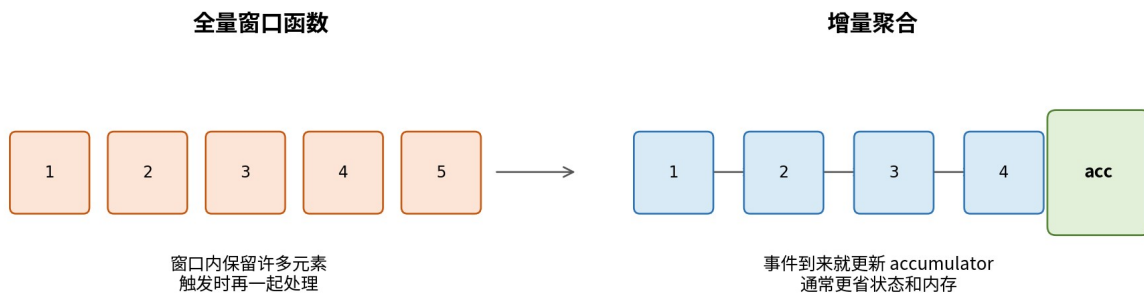
一个时间窗口通常在第一条属于它的事件到达时创建状态。Watermark 越过窗口结束时间时，默认 Event-Time Trigger 首次计算；如果配置 Allowed Lateness，窗口状态还会继续保留一段时间，以接收“迟到但仍允许”的事件。

阶段	发生什么
第一条事件进入	创建该 key + window 对应的状态
窗口期间	持续加入元素或更新增量 accumulator
Watermark 越过窗口末端	默认 EventTimeTrigger 触发计算
Allowed Lateness 期间	部分迟到事件仍可加入，并可能再次触发更新
Watermark 越过 end + lateness	窗口状态清理；更晚数据通常被丢弃或送 Side Output

默认行为：官方文档中，Event-Time Window 的 allowed lateness 默认是 0。也就是说，Watermark 已经越过窗口结束时间后才到达的数据，默认会被视作太晚。

7. Window Function：窗口里面到底怎么算？

窗口的最后一步是计算函数。最常见的两类思路是：全量窗口计算，以及增量聚合。



ReduceFunction / AggregateFunction 常用于增量聚合；ProcessWindowFunction 可获得窗口元数据

图 5：全量窗口函数与增量聚合的差别。

7.1 全量窗口函数

思路是把窗口中的许多元素保留下来，Trigger 触发时把这些元素交给 ProcessWindowFunction 等窗口函数。优点是逻辑灵活、能访问窗口起止时间等元数据；缺点是状态可能比较大。

7.2 增量聚合

ReduceFunction / AggregateFunction 可以在事件到来时就更新一个较小的 accumulator。例如求销售额，只需要保存 running sum，而不是保存窗口内所有订单。

同一问题的两种状态方式

订单金额：100，80，60，40

全量思路：状态里保存 [100，80，60，40]

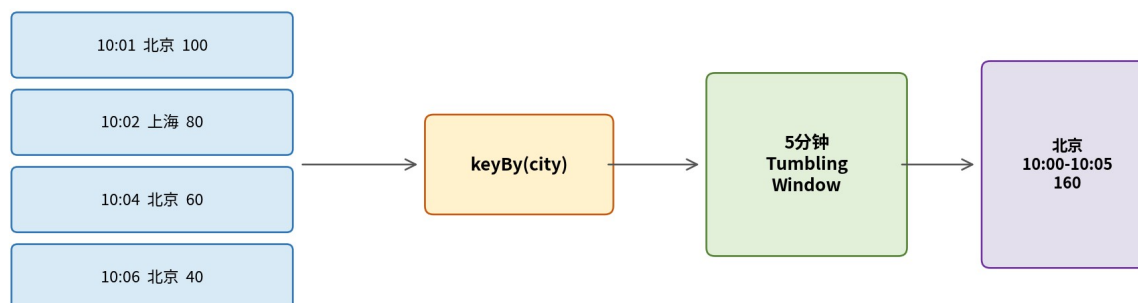
增量思路：状态只不断更新 sum = 280

性能原则：能用可合并的增量状态表达的问题，通常优先考虑增量聚合；只有确实需要访问窗口内完整元素或复杂上下文时，再保留更多数据。

Flink 还支持把 AggregateFunction/ReduceFunction 与 ProcessWindowFunction 组合：前者负责高效增量聚合，后者在触发时读取聚合结果并使用窗口元数据。

8. 完整例子：每个城市每 5 分钟销售额

例：每个城市每5分钟销售额



Watermark 越过 10:05 → EventTimeTrigger 触发 → 输出第一个窗口结果；10:06 的订单属于下一个窗口

图 6：一个完整 Event-Time Tumbling Window 流程。

业务需求：实时计算每个城市每 5 分钟的已支付订单销售额。假设 Event Time 已提取并已经有 Watermark。

1. Source 持续读入订单。
2. keyBy(city)：北京订单和上海订单进入各自 Keyed State 空间。
3. TumblingEventTimeWindow(5 分钟)：按 Event Time 决定所属时间窗口。
4. AggregateFunction：订单到达时增量累加 amount。
5. Watermark 越过 10:05：EventTimeTrigger 触发 [10:00,10:05) 窗口。
6. 输出北京 160、上海 80。10:06 的北京订单属于下一个窗口。

Event Time	城市	金额	窗口
10:01	北京	100	10:00-10:05
10:02	上海	80	10:00-10:05
10:04	北京	60	10:00-10:05
10:06	北京	40	10:05-10:10

关键观察：keyBy 决定“按谁分别算”；Window Assigner 决定“哪个时间段”；AggregateFunction 决定“怎么算”；Watermark + Trigger 决定“什么时候可以输出”。

9. Sliding Window：为什么一条事件可能被算多次？

假设业务要求：“每 1 分钟更新一次最近 5 分钟订单额”。这就是 size=5min、slide=1min 的 Sliding Window。

10:04:30 的一条订单可能同时属于多个窗口，例如 10:00-10:05、10:01-10:06、10:02-10:07、10:03-10:08、10:04-10:09。它并不是被“复制成 5 份业务数据”，而是在窗口语义上参与多个结果。

成本：滑动窗口越密，事件参与的窗口越多，状态与计算压力越高。因此“每秒更新最近 24 小时指标”这种需求必须谨慎设计。

10. Session Window：为什么窗口会合并？

假设 session gap = 30 分钟。用户在 10:00、10:10、10:25 都有行为，这些事件之间没有超过 30 分钟的空闲，因此属于同一次会话；如果下一次事件是 11:10，就会形成新会话。

Flink 内部会根据事件不断创建/合并候选 Session Window，因此 Session Window 需要支持窗口合并的 Trigger 和 Window Function。

场景	更适合的窗口
每小时报表	Tumbling
每分钟刷新最近 10 分钟指标	Sliding
用户一次访问/设备一次活跃周期	Session
没有固定边界、自定义复杂规则	自定义 Window / State+Timer（进阶）

11. keyBy().window() 与 windowAll() 的区别

在现实任务中通常先 keyBy 再 window，因为这样每个 Key 可以独立并行计算。若直接对整个流做一个全局窗口，就会失去按 Key 的天然并行性，容易形成单点瓶颈。

```
# 每个城市独立做窗口
stream.key_by(city).window(...).aggregate(...)

# 整条流作为一个整体窗口（概念）
stream.window_all(...).aggregate(...)
```

并行性原则：“先 keyBy 再 window” 不仅是业务分组，也是分布式并行策略。没有 keyBy 的全局窗口需要格外关注吞吐瓶颈。

12. 常见误区

误区 1：Watermark 决定 5 分钟一个窗口。

→ 错误。5 分钟边界由 Window Assigner 决定；Watermark 只是时间进度信号。

误区 2：窗口结束时间一到，系统立刻计算。

→ Event Time 模式下看的是 Watermark 是否推进到相应位置，而不是机器墙上时间。

误区 3：Trigger FIRE 后窗口一定立即消失。

→ 不一定。FIRE 与 PURGE 是不同动作，Allowed Lateness 也可能要求状态继续保留。

误区 4：Sliding Window 中一条事件只能属于一个窗口。

→ 错误。重叠 Sliding Window 中，一条事件可能参与多个窗口结果。

误区 5：Window 就是把所有元素存成 List。

→ 不一定。增量聚合可以只维护很小的 accumulator。

误区 6：Allowed Lateness 越大越好。

→ 越大意味着结果等待/更新更久、窗口状态保留更久，状态成本会上升。

误区 7：Session Window 就是固定 30 分钟窗口。

→ 不是。30 分钟通常是“不活动间隔”，会话实际长短由事件行为决定。

13. 小实验：手工模拟窗口

下面的 Python 代码不依赖 Flink，只用来训练 Window Assigner 的思维。给定事件分钟数，计算它属于哪个 5 分钟 Tumbling Window。

```
def tumbling_window(minute, size=5):
    start = (minute // size) * size
    end = start + size
    return start, end

for minute in [1, 4, 5, 9, 10, 14]:
    print(minute, "->", tumbling_window(minute))

# 预期:
# 1  -> (0, 5)
# 4  -> (0, 5)
# 5  -> (5, 10)
# 9  -> (5, 10)
```

再想一步：如果是 size=10、slide=5 的 Sliding Window，同一分钟可能落入几个窗口？这正是下一步理解窗口状态量的基础。

14. 练习题与答案

练习 1

业务要求：“每 10 秒输出一次最近 1 分钟 API 错误数”。应该用 Tumbling 还是 Sliding？

答案：Sliding Window。窗口长度 1 分钟，slide 10 秒。

练习 2

Event-Time Window 是 [10:00,10:05)，Watermark 已越过 10:05。一条 Event Time=10:04:50 的事件现在才到达。它一定被丢弃吗？

答案：不一定。要看 Allowed Lateness 和迟到数据处理策略。默认 allowed lateness=0 时通常太晚；若配置了容忍迟到时间，则可能仍进入窗口并再次触发结果。

练习 3

为什么求“窗口销售额”通常更适合 AggregateFunction，而不是把全部订单一直保存在窗口里？

答案：因为销售额可以用 running sum 增量维护，只保存 accumulator 即可，状态更小、内存压力更低。

练习 4

用户事件 10:00、10:12、10:35，Session gap=20 分钟。会形成几个 Session？

答案：2 个。10:00 与 10:12 间隔 12 分钟，属于同一会话；10:12 到 10:35 间隔 23 分钟，超过 20 分钟，开启新会话。

15. 把本讲压缩成 8 句话

7. 无界流没有天然结尾，所以需要 Window 把问题切成有限范围。
8. Tumbling 固定且不重叠；Sliding 可以重叠；Session 由不活跃间隔决定边界。
9. Window Assigner 决定事件属于哪个窗口。
10. Watermark 表示 Event Time 的推进进度，不负责定义窗口大小。
11. Trigger 决定窗口什么时候计算；Window Function 决定怎么算。
12. 窗口需要 State 保存元素或增量 accumulator。
13. Allowed Lateness 决定 Watermark 越过窗口末端后还愿意等多久。
14. 真正高效的窗口设计，本质是“正确的时间语义 + 合理的状态大小 + 可控的触发频率”。

下一讲：Flink 第 5 讲将进入 State：为什么流处理系统必须“记住过去”，Keyed State、Operator State、ValueState/ListState/MapState 到底是什么，以及状态如何和 keyBy 对齐。

资料核对（Apache Flink 官方文档）

本讲窗口类型、Trigger、Allowed Lateness、窗口生命周期和 PyFlink 窗口 API 均按 Apache Flink 官方 stable 文档核对。重点参考：Windows / 窗口、PyFlink TumblingEventTimeWindows、EventTimeSessionWindows、WindowedStream.aggregate。

注：Flink 的新 DataStream API V2 仍在演进中。本课程继续用跨版本更稳定的经典 DataStream 窗口心智模型讲原理；具体 API 名称以实际安装版本官方文档为准。